

Глубокий разбор CyOps Arena и лучший проект на победу для вашей команды

Executive summary

CyOps x MiniMax M3 Hackathon — это не обычный “собери стартап за выходные” хакатон, а соревнование, где публично заявлено, что **AI-судьи оценивают repository evidence**, а **комьюнити голосует за презентацию**; публичная страница арены также указывает **композитный итоговый счёт 50% AI judges + 50% community votes**. По публичным официальным материалам сейчас видны: **build week 10–16 июня 2026, submission deadline 16 июня, community voting и winner announcement 19 июня, призовой пул \$5,000 на 10 победителей**, а также то, что проекты будут продвигаться в каналах CyOps, Cysic и MiniMax. ¹

Из этого следует неприятная, но полезная правда: **побеждает не “умная идея”, а проект, который одновременно легко считывается моделью как сильный репозиторий и легко считывается людьми как эффектная история**. То есть нужен не “AI-ассистент вообще”, а артефакт с очень плотной доказательной базой: зелёный CI, фикстуры, before/after, воспроизводимый demo flow, явный README, отдельный файл для судей и ясный narrative. Публичная submit-страница треков буквально подталкивает к этому: в одних треках важны рабочий PR и measurable before/after, в других — story “this thing was dead, and now it works”. ²

Мой лучший и самый реалистичный выбор для вас двоих — `Lazarus MCP`: MCP-сервер и CLI для “вскрытия” и оживления мёртвых Node/Python-репозитория, который запускает автопроверку, находит причину смерти, накатывает безопасные **playbook-фиксы**, повторно **прогоняет build/test** и генерирует **machine-readable judge pack** с `RESURRECTION_REPORT.md`. В submit-форме я бы шёл в **трек The Resurrection** как основной, а “MCP server” использовал как техническое усиление и маркетинговую упаковку, а не как основной трек. Это даёт лучший баланс между шансом реально зашипить и шансом выиграть голоса. ³

Важно ещё одно: **публично доступных recent winning submissions у этого хакатона сейчас нет**. На арене в открытом доступе ещё нет видимого списка проектов и победителей, а winner announcement указан только на **19 июня 2026**; публичный leaderboard в доступной разметке пока пустой. Значит, нельзя честно “скопировать победителя”; оптимальная стратегия строится по официальным правилам, трекам и механике оценки, а не по зафиксированным кейсам прошлых победителей. ⁴

Официальная механика хакатона и что она реально означает

Публичная submit-страница требует для сабмита **linked Cysic AI account** и **verified wallet**; вход допускается через **X или Discord**. На этой же странице перечислены четыре рекомендуемых трека: **Ship-a-Feature**, **MCP Server Sprint**, **Whole-Repo Refactor** и **The Resurrection**. При этом сама страница в терминах треков не говорит абстракциями: для Ship-a-Feature — рабочий PR по реальному issue и бонус за merge; для MCP Server Sprint — сервер, который другие разработчики

реально захотят установить; для Whole-Repo Refactor — measurable before/after; для The Resurrection — сильная история “было мёртвым, теперь работает”. ⁵

Официальная продуктовая документация CyOps подтверждает, что хакатон вообще заточен не под “IDE с автокомплитом”, а под **web-based orchestration harness** для **multi-agent coding workflows** между агентами вроде **Claude Code, Codex и OpenCode**. Документация также подчёркивает execution plan, review/feedback loop, observability, workspace and Git и полную видимость цикла выполнения. Это значит, что выигрывает не просто код, а **структурированная инженерная работа, которую удобно читать и проверять как людям, так и AI-судье**. ⁶

Публичная tutorial-документация CyOps полезна и для тактики. В ней явно есть: **генерация плана**, отдельный файл `plan.md`, **status tab** с run status, iteration, strategy, phase и acceptance progress, единый **workspace** на весь project, GitHub connection из Git tab/Settings и несколько web terminals. Это важно не только для удобства разработки, но и для вашей доказательной базы: план, acceptance criteria и артефакты уже встроены в сам рабочий процесс CyOps. ⁷

Публичные официальные сниппеты по арене дают ключевые даты и механику. Они указывают **June 10–16 Build Week, daily progress sharing on X, June 16 (EOD) Submission Deadline, demo verification submitted**, затем **public gallery live, community voting, June 19 Voting Deadline** и **June 19 Winner Announcement**. Отдельные официальные посты Cysic дополнительно фиксируют **\$5,000 prize pool, 10 winners, 80% off MiniMax M3 model token pricing**, а также диапазон вознаграждений **\$200–\$400** для winning projects. ⁸

Отдельный operational caveat: официальный launch-snippet Cysic также говорил, что **applications close June 8, 11:59 PM UTC**. Публичная арена, однако, всё ещё показывает sign-in/profile flow. Если вы **уже внутри арены**, для вас критичен дедлайн 16 июня. Если **нет**, нужно немедленно проверить в профиле/Discord, допускается ли late entry; публичные материалы здесь не вполне консистентны. ⁹

Ниже — сжатая сводка по официальной механике. Она основана на официальной арене, официальных docs Cysic и официальных постах Cysic/MiniMax. ¹⁰

Параметр	Что подтверждено публично	Практический смысл для вас
Формат	Global online hackathon	Можно полностью делать всё дистанционно
Сроки	Build Week 10–16 июня, дедлайн сабмита 16 июня, голосование/ анонс 19 июня	Нужно финишировать технику к 15 июня, а 16-е оставить под упаковку
Призы	\$5,000 на 10 winners, \$200–\$400 на проект	Конкуренция не окупает “долгую фундаментальную платформу”; нужен быстрый, зрелищный win
Оценка	50% AI judges + 50% community votes	Нужны одновременно repo evidence и сильная презентация
Голосование	У всех 2 голоса; у Cysic AI users +1	Название, однострочный питч и видео важны почти так же, как код

Параметр	Что подтверждено публично	Практический смысл для вас
Сабмит	Нужны linked Cysic AI account и verified wallet	Не откладывайте профиль и wallet verification
Треки	Ship-a-Feature, MCP Server Sprint, Whole-Repo Refactor, The Resurrection	Наиболее выгоден гибрид MCP + Resurrection, но submit лучше делать как Resurrection

Ещё один важный нюанс: на публичной submit-странице виден заголовок **AI Score rubric**, но сама доступная страница сейчас показывает лишь **“Loading rubric...”**. Поэтому подробные sub-weights внутри AI score публично не подтверждены. Честный вывод такой: **детальный подрубриктор сейчас недоступен**, а значит трек-описания и публичная 50/50-механика — это ваш фактический north star. ¹¹

Как здесь реально выиграть, а не просто поучаствовать

Ваша победная стратегия должна быть циничной и очень конкретной. Раз AI judges читают **repository evidence**, проект нужно строить так, будто его первым пользователем будет не человек, а модель, которая открывает корень репозитория и пытается понять за 30–90 секунд: **что это, работает ли это, где доказательства, в чём before/after**. Раз community votes завязаны на presentation и у каждого мало голосов, история должна быть мгновенно понятной и визуально резкой. “AI toolchain for developers” — плохо. “Мы подняли труп репозитория из могилы за 4 минуты” — хорошо. ¹²

Это означает, что **generic AI wrapper** — слабая идея. Если вы принесёте “ещё один MCP-server для всего на свете”, он утонет по двум причинам. Во-первых, MCP Sprint прямо требует **useful MCP server for real developer workflows** и даже упоминает бонус за traction during the hackathon; без конкретного узкого workflow это выглядит как пустая обвязка. Во-вторых, AI-судья не сможет извлечь из такого репозитория сильный before/after narrative. ¹¹

Гораздо сильнее работает **узкая победная wedge-идея**, у которой одновременно есть инженерная ценность и театральная подача. Для этого конкретного хакатона лучшая wedge — **repo resurrection**. Трек The Resurrection буквально под это написан: abandoned, broken or half-finished project; compelling narrative; now it works. Это уже наполовину готовый pitch. Если вы ещё и завернёте это в MCP/CLI-инструмент, то получите не просто историю, а переиспользуемый dev-tool, что усиливает техническую “серьёзность” проекта. ¹³

Ниже — сравнение четырёх реалистичных идей. Оценки **win probability / effort / judge-fit / novelty** — это аналитическое суждение на основе официальных треков, 50/50 scoring и текущей фазы хакатона, а не официальные баллы. ¹¹

Идея	Лучший трек	Win-probability	Инженерное усилие	Judge-fit	Новизна	Главный риск
Lazarus MCP — оживление мёртвых Node/Python-репо с evidence pack	The Resurrection	9/10	7/10	10/10	8/10	Перескопить в “универсальный автофикс для всего”
Issue Hunter — агент, который берёт GitHub issue и готовит PR	Ship-a-Feature	6/10	7/10	8/10	6/10	Merge зависит от внешних maintainer’ов
Repo Modernizer — массовая миграция whole repo	Whole-Repo Refactor	5.5/10	8.5/10	8/10	5/10	Нужен большой кодбейс и железный before/after
On-call / Observability MCP	MCP Server Sprint	5/10	8/10	6.5/10	7/10	Съедят OAuth, API-auth и интеграции

Слабые места, на которых команды тут почти наверняка будут погибать, тоже видны заранее. Ниже — не “общие советы”, а прямые ямы, вытекающие из опубликованных правил и интерфейса арены. ¹⁴

Ошибка	Почему это убивает шанс на победу	Что делать вместо этого
Делать стартап-дек и красивый UI без плотного репо	AI judges оценивают repository evidence	Сначала ядро, тесты, отчёт, CI; UI — потом
Заявить “поддерживаем все языки”	Сломает срок и надёжность	Ограничиться Node + Python в MVP
Завязать успех на merge от maintainer’a	Это внешняя зависимость, которую вы не контролируете	Делать проект, где основное доказательство находится внутри вашего репо
Не оставить machine-readable summary	AI-судье сложнее распарсить сильные стороны	Положить в корень <code>AI_JUDGES.md</code> , <code>RESURRECTION_REPORT.md</code> , <code>evidence/summary.json</code>

Ошибка	Почему это убивает шанс на победу	Что делать вместо этого
Не думать о голосах до конца	У каждого лишь 2–3 голоса; они уйдут проекту с ясным one-liner	Нужен мэмный, мгновенно понятный pitch и 60–90-секундное видео
Тянуть с profile/wallet verification	Можно банально не успеть сабмитнуть	Закрыть аккаунт и wallet в первый же час

Самый важный спорный вывод такой: **submit лучше делать не в MCP Server Sprint, а в The Resurrection**, даже если у вас технически MCP-сервер. Почему? Потому что MCP Sprint публично намекает ещё и на “traction during the hackathon window”, а traction за несколько дней без готовой аудитории — вещь капризная. The Resurrection, наоборот, награждает именно то, что вы реально можете контролировать: “было мёртвым — стало живым”. В описании проекта всё равно можно подчеркнуть, что это ещё и MCP-server other developers can install. ⁵

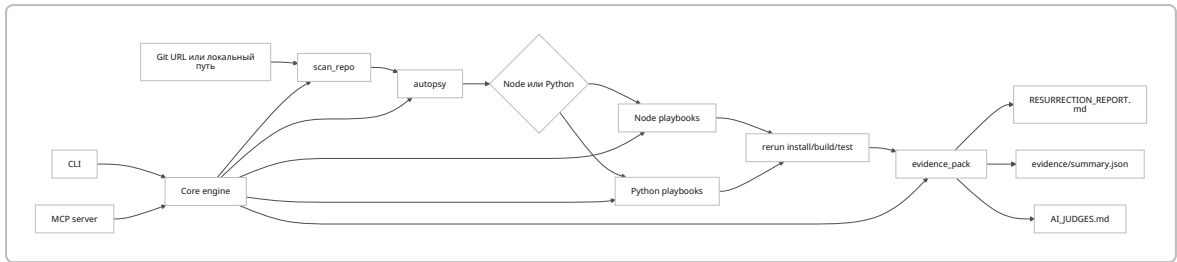
Лучший проект на победу

Рекомендация: `Lazarus MCP` — “autopsy-and-resurrection engine” для сломанных Node/Python-репозиториях. Пользователь даёт локальный путь или git URL, сервер/CLI определяет стек, запускает install/build/test, собирает “протокол смерти”, применяет безопасные playbook-фиксы, повторно гоняет пайплайн и выпускает отчёт, который легко читать человеку и AI-судье. Суть не в магическом AGI-ремоне, а в **очень убедительном, доказуемом потоке автодиагностики и частичного/полного восстановления**. Это критично: обещать “чинит любой репозиторий” — бред; обещать “быстро поднимает класс распространённых Node/Python-поломок и документирует результат” — сильно и правдоподобно. ¹³

Почему именно он оптимален. Во-первых, он **идеально попадает в механику The Resurrection** и при этом захватывает технический престиж **MCP**. Во-вторых, он очень хорошо ложится на **MiniMax M3**, который официально позиционируется как модель для coding, agentic work, tool use и long-context tasks с окном до **1,000,000 токенов**; это полезно, когда надо читать кодовую базу, логи, конфиги и build output как единую задачу. В-третьих, его очень удобно демонстрировать в 60–90 секунд: “вот broken repo, вот autopsy, вот fix plan, вот снова зелёный build/test, вот отчёт”. Это очень сильный community-vote asset. ¹⁵

Мой жёсткий совет по scope: **делайте только Node и Python**. Не трогайте Rust, Go, Java, mobile и container orchestration beyond basics в MVP. Срок хакатона слишком короткий, а официальная механика награждает не breadth, а доказуемый shipped result. Если вы попытаетесь поддерживать всё, итогом будет недоделанная оболочка без сильного кейса. Если вы ограничитесь Node/Python, вы успеете сделать детект, playbook-фиксы, фикстуры, тесты, отчёты и одно внешнее case study. На такой дистанции это гораздо выигрышнее. ¹⁶

Ещё один важный ход: **архитектура должна быть CLI-first, а MCP — тонким адаптером**. Официальные MCP docs разделяют tools/resources/prompts и поддерживают как **stdio** для локальных process-spawned интеграций, так и **Streamable HTTP** для remote servers; remote transport даже называется рекомендованным для удалённых серверов, но для вашего MVP это лишняя поверхность риска. Жюри смотрит repo evidence, а не production hosting. Поэтому правильный путь — сначала чистое ядро на TypeScript и CLI, потом MCP-обёртка поверх него. ¹⁷



Если в submit-форме можно выбрать только один трек, мой выбор — **The Resurrection**. Если есть поле “description”, в первой же строке пишете: **“A reusable MCP + CLI engine that resurrects dead Node/Python repos and generates judge-ready evidence.”** Так вы одновременно бъёте в narrative и в dev-tool utility. ¹¹

Полная спецификация MVP и что именно должно увидеть жюри

Минимально жизнеспособный продукт должен состоять не из “AI-мозга”, а из четырёх жёстких функций: `scan_repo`, `autopsy`, `resurrect`, `evidence_pack`. Этого достаточно, чтобы сделать полноценный pipeline и сильный демо-кейс. MCP официально крутится вокруг **tools/resources/prompts**, а МЗ официально заточен под tool use, coding и long-context tasks; это хорошо сочетается с вашей задачей, но не требует экспериментальных фич. Для MVP вам не нужны ни long-running tasks API, ни remote auth, ни OAuth. ¹⁸

Компонент	Что делает	Вход	Выход
<code>scan_repo</code>	Детектит стек, менеджер пакетов, точки входа, тестовый стек, lockfiles, CI-файлы, уровень риска	path / git URL	<code>stack</code> , <code>pkgManager</code> , <code>commands</code> , <code>healthHints</code>
<code>autopsy</code>	Прогоняет install/build/test/lint в sandbox-safe режиме, собирает structured logs и классифицирует причину падения	path / git URL	<code>failStage</code> , <code>errorClass</code> , <code>suggestedPlaybooks</code> , <code>beforeMetrics</code>
<code>resurrect</code>	Накатывает безопасные playbook-фиксы для Node/Python, создаёт локальную ветку, сохраняет дифф	path / git URL, режим safe	<code>appliedFixes</code> , <code>afterMetrics</code> , <code>gitDiffPath</code> , <code>branchName</code>
<code>evidence_pack</code>	Генерирует judge-facing отчёт и machine-readable summary	результаты всех шагов	<code>RESURRECTION_REPORT.md</code> , <code>AI_JUDGES.md</code> , <code>evidence/summary.json</code>

По стеку я бы взял **TypeScript + Node 20+**, потому что официальные quickstart-доки MCP используют именно этот путь, прямо указывают **Node.js 20 or higher**, работу через `McpServer`,

`StdioServerTransport`, `zod`, а также подчёркивают, что в stdio-based MCP servers надо логировать через `console.error`, а не `console.log`, чтобы не ломать JSON-RPC канал. ¹⁹

По SDK я бы принял прагматичное решение. Официальный сайт SDK прямо говорит, что ветка **v2 currently in development, pre-alpha**, а **v1.x remains the recommended version for production use**. Но текущие официальные examples для серверов уже показывают split-packages вроде `@modelcontextprotocol/server`. Из-за этого вам не надо строить архитектуру вокруг версии SDK: делайте **core engine отдельно**, а MCP adapter — тонким слоем, который можно быстро переключить. Это реальный способ снять риск документационной турбулентности и не потерять сутки на миграции. ²⁰

Рекомендуемая структура репозитория должна быть предельно “судейской”: сильные файлы — в корне, функциональность — в `src`, доказательства — в `evidence`, готовые demo cases — в `fixtures` и `cases`. Это не косметика. Поскольку public arena явно говорит, что AI judges смотрят repository evidence, удобная корневая навигация — это прямой способ поднять шанс, что модель увидит ваши сильные стороны, а не потеряется в папках. Это уже не вкусовщина, а optimisation under rubric. ²¹

```
lazarus-mcp/  
  README.md  
  AI_JUDGES.md  
  RESURRECTION_REPORT.md  
  package.json  
  tsconfig.json  
  .gitignore  
  .github/workflows/ci.yml  
  
src/  
  cli.ts  
  server.ts  
  types.ts  
  core/  
    detect.ts  
    runner.ts  
    git.ts  
    logs.ts  
    reports.ts  
    safety.ts  
  tools/  
    scanRepo.ts  
    autopsy.ts  
    resurrect.ts  
    evidencePack.ts  
  playbooks/  
    node/  
      detectNodeIssues.ts  
      patchPackageJson.ts  
      patchEsmCjs.ts  
      patchMissingDeps.ts  
    python/
```

```

    detectPythonIssues.ts
    patchPyproject.ts
    patchRequirements.ts
    patchPytestConfig.ts

evidence/
  summary.json
  before-after.json

fixtures/
  dead-node-esm-cjs/
  dead-node-missing-build/
  dead-python-missing-pytest/

cases/
  external-case-study/
    BEFORE.md
    AFTER.md

docs/
  DEMO_SCRIPT.md
  VIDEO_STORYBOARD.md
  PR_BODY.md

```

Тесты должны быть трёх типов. Первый — unit tests на детекторы и классификаторы ошибок. Второй — fixture tests, которые поднимают ваши специально сложенные мини-репозитории и проверяют, что `resurrect` их реально чинит. Третий — golden tests на `RESURRECTION_REPORT.md` и `evidence/summary.json`, чтобы judge-facing артефакты не деградировали. Если у вас не будет именно fixture-based end-to-end тестов, проект будет выглядеть как исследовательский прототип, а не как shipped tool. ¹¹

CI должен быть скучным и жёстким: **install → build → test → typecheck**. Не нужно раздувать его matrix'ами. Вам важен не enterprise-CI, а зелёный и быстрый badge, который AI и люди увидят в README. Отдельная задача CI — гонять fixture repos и проверять, что `evidence_pack` всегда генерируется. Это будет намного сильнее любого красивого front-end. ²¹

Демо-план должен быть буквально театрально простым. Кадр 1: broken repo, красный install/test. Кадр 2: `lazarus autopsy`, видна причина. Кадр 3: `lazarus resurrect`, видны применённые playbooks. Кадр 4: rerun green. Кадр 5: открыт `RESURRECTION_REPORT.md` с before/after. Кадр 6: открыт `AI_JUDGES.md` с картой доказательств. Такой ролик отлично бьёт одновременно в community votes и в narrative The Resurrection. ²²

Самое важное, что именно нужно показать судьям, — это не абстрактный список features, а **доказательная последовательность**. Ниже — минимальный judge pack, который я считаю обязательным. Это уже аналитическая рекомендация, выведенная из того, что публично известно о 50/50 scoring и repository evidence. ²¹

Артефакт	Зачем он нужен	Статус
<code>README.md</code> с one-liner и quickstart	Человек и модель должны мгновенно понять проект	Обязательно
<code>AI_JUDGES.md</code>	Явная карта репозитория для AI judge	Обязательно
<code>RESURRECTION_REPORT.md</code>	Главный before/after narrative	Обязательно
<code>evidence/summary.json</code>	Machine-readable краткое резюме	Обязательно
Зелёный CI	Самый дешёвый сигнал “это работает”	Обязательно
Фикстуры сломанных репо + e2e tests	Доказывают воспроизводимость	Обязательно
Один внешний case study	Показывает, что это не только демо-игрушка	Очень желательно
60–90s demo video	Нужен для community votes	Обязательно
Скрин/лог с красным “до” и зелёным “после”	Самый сильный визуальный якорь	Обязательно

Пакет запуска через Codex прямо сейчас

Если начинать **сегодня, 12 июня 2026**, то первый Codex-run нельзя тратить на лендинг, картинки и “vision-doc”. Public rules уже сказали вам, что проект будет считан через **repo evidence** и **presentation**; следовательно, сначала надо заложить сильное репо, а презентация уже ляжет сверху. Самый правильный первый шаг — **за один проход Codex** создать CLI-first skeleton, фикстуры, тесты, CI, отчётный шаблон и minimal MCP adapter. ²¹

Ниже — **один самодостаточный файл** в формате **Markdown multi-message prompt file**. Его идея простая: вы вставляете содержимое целиком в Codex как initial prompt. Я специально сделал его не “поэтическим”, а командным: что создать, что имплементировать, какие команды гнать, какие ограничения соблюдать, какие outputs вернуть.

```
<!-- FILE NAME: CODEX_BOOTSTRAP.md -->
# SYSTEM

You are Codex acting as a senior TypeScript/CLI engineer and pragmatic
hackathon cofounder.
Your job is to scaffold and implement a working MVP repository named
`lazarus-mcp`.
Optimize for:
1. green build + tests,
2. reproducible demo,
3. judge-readable repository evidence,
4. minimal moving parts,
5. safe local git workflow.
```

Be decisive. Prefer a narrower working scope over a broad unfinished scope.
Do not build a frontend unless explicitly required later.
Do not over-engineer abstractions.
Do not block on remote deployment.
If MCP adapter work becomes unstable, keep the CLI/core fully working and make the MCP layer a thin adapter.

DEVELOPER

Project concept:

Lazarus MCP is a CLI-first TypeScript project plus MCP adapter that:

- scans a local or cloned repo,
- runs an autopsy on install/build/test,
- applies safe resurrection playbooks for common Node/Python issues,
- reruns the pipeline,
- generates a root-level `RESURRECTION_REPORT.md`,
- generates `AI_JUDGES.md`,
- generates `evidence/summary.json`.

Exact MVP scope:

- Support ONLY Node and Python repos.
- Support local path input first.
- Optional git URL cloning is welcome but not required for the first green build.
- No Rust/Go/Java/mobile support in MVP.
- No remote HTTP server in MVP.
- stdio MCP adapter is enough.
- No experimental task API in MVP.
- No OAuth/auth/web UI in MVP.

Repository requirements:

Create these files and directories:

```
lazarus-mcp/  
  README.md  
  AI_JUDGES.md  
  RESURRECTION_REPORT.md  
  package.json  
  tsconfig.json  
  .gitignore  
  .github/workflows/ci.yml
```

```
src/  
  cli.ts  
  server.ts  
  types.ts  
  core/  
    detect.ts  
    runner.ts  
    git.ts  
    logs.ts
```

```

    reports.ts
    safety.ts
    metrics.ts
tools/
    scanRepo.ts
    autopsy.ts
    resurrect.ts
    evidencePack.ts
playbooks/
    node/
        detectNodeIssues.ts
        patchPackageJson.ts
        patchEsmCjs.ts
        patchMissingDeps.ts
    python/
        detectPythonIssues.ts
        patchPyproject.ts
        patchRequirements.ts
        patchPytestConfig.ts

tests/
    detect.test.ts
    autopsy.test.ts
    resurrect.node.test.ts
    resurrect.python.test.ts
    report.test.ts

fixtures/
    dead-node-esm-cjs/
    dead-node-missing-build/
    dead-python-missing-pytest/

evidence/
    .gitkeep

docs/
    DEMO_SCRIPT.md
    VIDEO_STORYBOARD.md
    PR_BODY.md

```

Tech stack:

- Node 20+
- TypeScript
- npm
- vitest
- zod
- tsx
- execa or child_process wrapper
- simple-git only if helpful
- minimal dependencies
- if MCP package choice is ambiguous, prefer the currently installable

official package layout for server + stdio.

- keep all core logic package-agnostic so MCP layer can be swapped later

Public APIs to implement:

CLI commands:

- ``lazarus scan <path>``
- ``lazarus autopsy <path>``
- ``lazarus resurrect <path> [--safe] [--branch <name>]``
- ``lazarus evidence-pack <path>``

MCP tools:

- ``scan_repo``
- ``autopsy``
- ``resurrect``
- ``evidence_pack``

Core behavior:

- - ``scan_repo`` returns detected language stack, package manager, config files, scripts, test runner hints, and confidence.
 - ``autopsy`` executes safe commands with timeout and returns:
 - detected phase failures,
 - categorized error classes,
 - normalized logs,
 - suggested playbooks.
 - ``resurrect``:
 - creates local branch ``feat/lazarus-mvp`` or provided branch name,
 - applies safest possible fixes,
 - reruns install/build/test,
 - records before/after metrics,
 - never pushes to upstream.
 - ``evidence_pack`` creates:
 - root ``RESURRECTION_REPORT.md``
 - root ``AI_JUDGES.md``
 - ``evidence/summary.json``

Required error classes:

- `missing_dependency`
- `missing_script`
- `esm_cjs_mismatch`
- `missing_test_dependency`
- `unsupported_stack`
- `command_timeout`
- `unknown_failure`

Required Node resurrection playbooks:

- determine package manager from lockfile/packageManager field
- patch missing build/test scripts when clearly inferable
- patch common ESM/CJS mismatch cases conservatively
- install missing dev deps only when project structure clearly requires them
- rerun pipeline after each applied playbook

- stop if fix confidence is low

Required Python resurrection playbooks:

- detect pyproject.toml / requirements.txt / pytest usage
- add missing test dependency only when config/tests clearly imply pytest
- patch minimal packaging/test config issues conservatively
- rerun pipeline
- stop if fix confidence is low

Fixture repos to create:

1. ``fixtures/dead-node-esm-cjs``
 - create a tiny Node project with a deterministic ESM/CJS mismatch
 - include a failing build or test before patch
 - after patch, build/test must pass
2. ``fixtures/dead-node-missing-build``
 - create package.json with source files and tsconfig but missing build script
 - after patch, build must pass
3. ``fixtures/dead-python-missing-pytest``
 - create tiny Python package with tests and pytest config but missing pytest dependency
 - after patch, tests must pass

Testing requirements:

- add unit tests for detectors and log classifiers
- add end-to-end tests against all three fixtures
- add report generation snapshot/golden tests
- CI must run build + tests

README requirements:

- one-line pitch
- why this project exists
- quickstart
- CLI usage
- MCP tools
- demo flow
- current MVP scope: Node + Python only
- constraints and safety
- screenshots/log placeholders if no images exist yet

AI_JUDGES.md requirements:

- explain in one page:
 - what the project does,
 - where to find the main evidence,
 - which fixtures prove resurrection,
 - where before/after metrics live,
 - known MVP limitations.

RESURRECTION_REPORT.md requirements:

- problem statement
- before state

- autopsy findings
- applied fixes
- after state
- metrics
- changed files
- remaining limitations

``evidence/summary.json`` schema:

```
{
  "project": "lazarus-mcp",
  "version": "0.1.0",
  "mvp_scope": ["node", "python"],
  "fixtures": [
    {
      "name": "...",
      "before": {"install": "...", "build": "...", "test": "..."},
      "after": {"install": "...", "build": "...", "test": "..."},
      "appliedFixes": ["..."]
    }
  ],
  "externalCaseStudy": null,
  "knownLimitations": ["..."]
}
```

Command execution policy:

- timeout install/build/test commands after 120 seconds by default
- timeout autopsy on external repos after 180 seconds
- kill child processes cleanly
- capture stdout/stderr separately
- redact obvious secrets if encountered
- use ``console.error`` instead of ``console.log`` in stdio MCP mode if needed

Git safety policy:

- NEVER push to upstream
- NEVER force push
- NEVER modify git remotes unless user explicitly asks
- create only local branch
- NEVER run destructive commands outside the current repo
- do not delete files outside the project directory
- do not exfiltrate secrets or env vars into logs/reports

Resurrection safety policy:

- only apply high-confidence patches
- if confidence is low, generate a suggested fix section instead of mutating files
- keep applied changes minimal and auditable
- prefer smallest patch that turns red -> green
- preserve original failing logs in the report

Expected outputs before you stop:

1. repo scaffold exists

2. ``npm install`` completes
3. ``npm run build`` succeeds
4. ``npm test`` succeeds
5. all three fixture repos are used in tests
6. root ``README.md`` is strong
7. root ``AI_JUDGES.md`` exists
8. root ``RESURRECTION_REPORT.md`` exists
9. ``evidence/summary.json`` exists
10. a local branch named ``feat/lazarus-mvp`` exists, or exact commands to create it are printed if git is unavailable
11. ``docs/DEMO_SCRIPT.md``, ``docs/VIDEO_STORYBOARD.md``, ``docs/PR_BODY.md`` exist

Execution order:

1. initialize repo files
2. implement types + core utilities
3. implement ``scan_repo``
4. implement ``autopsy``
5. create fixtures
6. implement resurrection playbooks
7. implement ``evidence_pack``
8. implement CLI
9. implement MCP adapter
10. write tests
11. run install/build/test
12. fix failures until green
13. write docs
14. summarize exactly what was built, what remains, and which commands were used

When uncertain:

- choose the minimum viable working path
- keep scope narrow
- keep reports explicit
- optimize for a judge reading the repo cold

USER

Create the full ``lazarus-mcp`` MVP now.

Work directly in files.

Run the necessary commands.

Do not stop at scaffolding; make it pass build and tests.

If an MCP package mismatch slows you down, keep the CLI core green and implement the thinnest working MCP adapter possible.

Return:

- a concise build summary,
- the final file tree,
- test/build status,
- and remaining limitations.

Для итеративной работы после bootstrap лучше не “болтать с Codex”, а давать ему короткие, агрессивно конкретные задачи. Ниже — лучшие шаблоны, которые я бы использовал сразу после первого прохода.

Prompt для `scan_repo`:

```
Implement src/tools/scanRepo.ts and supporting detectors so that Lazarus can reliably detect:  
- Node vs Python  
- package manager / environment manager  
- scripts / test runner hints  
- confidence score  
Add unit tests for ambiguous repos and fixture repos.  
Keep output schema stable and machine-readable.  
Do not touch docs unless needed.  
Run tests for scan-related files only, then full test suite if green.
```

Prompt для `autopsy`:

```
Implement autopsy with structured error classification.  
Requirements:  
- run install/build/test safely with timeout  
- capture stdout/stderr separately  
- classify failures into the defined error classes  
- emit normalized logs and suggested playbooks  
- preserve raw failure excerpts in a safe report field  
Add tests covering timeout, missing dependency, missing script, and unknown failure.
```

Prompt для Node-репо:

```
Implement the Node resurrection playbooks for common recoverable failures:  
- missing build/test script when inferable  
- ESM/CJS mismatch in common config/entry scenarios  
- missing dev dependency when confidently implied by config  
Keep patches minimal and auditable.  
Add end-to-end tests using dead-node fixtures.  
Do not add speculative package changes.
```

Prompt для Python-репо:

```
Implement the Python resurrection playbooks for common recoverable failures:  
- detect pytest usage from tests/config  
- patch missing test dependency when confidence is high  
- minimally patch pyproject or requirements when necessary
```


Add end-to-end tests using dead-python fixture.
Keep the scope to pytest-based projects only.

Prompt для фикса CI:

Run the full build and test pipeline.
Diagnose every failure.
Patch the smallest possible surface area until CI is green.
Do not weaken tests.
If a test is flaky, make it deterministic instead of deleting it.
Update .github/workflows/ci.yml only if necessary.

Prompt для README и AI_JUDGES.md :

Rewrite README.md and AI_JUDGES.md for judges who will read the repo cold.
Requirements:
- first 15 seconds of reading must answer what/why/how/evidence
- add a clear quickstart
- add a section called "Proof it works"
- point explicitly to fixtures, tests, CI, report, and summary.json
Keep the tone sharp, technical, and concrete.

Prompt для RESURRECTION_REPORT.md :

Generate a judge-facing RESURRECTION_REPORT.md using the fixture results.
It must show:
- before state
- autopsy
- applied fixes
- after state
- metrics
- known limitations
Prefer tables and concise sections over long prose.

Prompt для 60-90s видео-сториборда:

Write docs/VIDEO_STORYBOARD.md for a 60-90 second demo video.
The video must show:
1. broken repo
2. autopsy
3. resurrect
4. green rerun
5. report
6. one-line closing pitch

Keep every shot short and visual.
Assume terminal + VS Code recording only.

Помпт для PR body:

Write docs/PR_BODY.md as if this were a hackathon submission/update PR.
Include:

- summary
- what works now
- evidence
- limitations
- next milestone

Tone: serious engineering, not hype.

Помпт для ESM/CJS-патча, когда Codex тупит:

I have a Node fixture failing because of an ESM/CJS mismatch.
Diagnose whether the least invasive fix is:

- renaming a config file to .cjs,
- switching to export default,
- changing package.json type,
- or adjusting the runner command.

Pick exactly one minimal fix, explain why, implement it, and add a regression test.
Do not make broad ecosystem migrations.

Помпт для dependency hell:

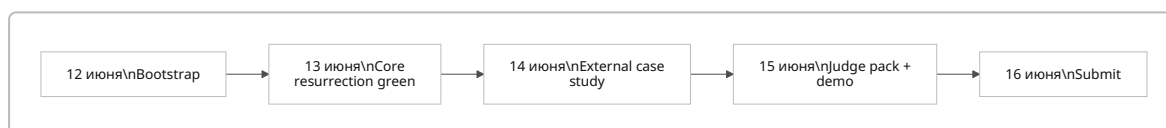
The repo is failing due to missing dependencies.
Only add dependencies that are strongly implied by source/config/scripts.
Show the evidence for each added package.
After patching, rerun the smallest relevant command first, then full test/build.
Output appliedFixes and rationale.

План на пять дней до дедлайна

Ниже — план, заточенный под текущую публичную фазу арены: **12 июня — уже середина build week**, а public snippets указывают дедлайн сабмита на **16 июня** и голосование/анонс на **19 июня**. Поэтому игра идёт не в “красивую платформу”, а в **жёсткое уплотнение шансов**: сначала core, потом evidence, потом story. ²³

Дата	Главная цель дня	Что должно быть готово к концу дня
12 июня	Bootstrap через Codex	Skeleton repo, CLI, фикстуры, <code>scan_repo</code> , <code>autopsy</code> , базовые тесты

Дата	Главная цель дня	Что должно быть готово к концу дня
13 июня	Реальное “воскрешение” фикстур	<code>resurrect</code> для Node/Python, зелёные e2e tests, <code>evidence_pack</code>
14 июня	Внешний case study	Один публичный stale repo или собственный “half-finished” проект, который реально поднят и задокументирован
15 июня	Упаковка для судей и голосов	<code>README</code> , <code>AI_JUDGES.md</code> , <code>RESURRECTION_REPORT.md</code> , демо-скрипт, сториборд, финальный CI
16 июня	Сабмит и presentation assets	Субмит на арене, финальное 60–90s видео, short pitch для X/Discord, превью-картинка и один скрин “до/после”



Практически это надо делать так. **12 июня** вы не строите product vision — вы добываете зелёный костяк. **13 июня** вы не добавляете новые языки — вы делаете так, чтобы три fixture-репо стабильно воскресали и генерировали отчёт. **14 июня** вы берёте один внешний case study, но только такой, который укладывается в Node/Python и в вашу existing playbook-модель. **15 июня** вы перестаёте “разрабатывать” и начинаете упаковывать репозиторий для холодного прочтения. **16 июня** — только submit, видео, social copy и sanity-check. Это и есть оптимальный режим под текущие официальные сроки. ²⁴

Публичные CyOps docs ещё и подсказывают, как вести саму работу внутри платформы. Для проекта создаётся отдельный workspace; после описания задачи можно сгенерировать initial implementation plan, потом просматривать/редактировать `plan.md`, следить за acceptance progress и использовать несколько терминалов. Если вы реально делаете run’ы в CyOps, фиксируйте acceptance criteria прямо в `plan.md` и держите GitHub подключённым — так у вас будет и техническая, и процессная “доказательная линия”. ²⁵

Так как публично подтверждены **2 голоса на человека** и ещё **+1 голос для Cysic AI users**, не тяните с social packaging. Вам нужен не длинный тред, а кристально ясный one-liner. Лучший вариант формулировки: **“Lazarus MCP autopsies dead repos, revives common Node/Python failures, and generates judge-ready proof.”** Это считается и как dev tool, и как resurrection story. А вот “agentic workflow orchestration for multi-environment software repair” звучит как бессмысленный туман и проигрывает. ²⁶

Официальные источники, на которые стоит опираться при сабмите и упаковке проекта, такие: **Cysic Arena home / leaderboard / submit** для механики хакатона и итогового scoring; **CyOps docs** и **How to Use CyOps** для workspace, планов, strategic profiles и GitHub flow; **MCP TypeScript SDK docs** и **server examples** для основы MCP-слоя; **MiniMax M3 official model page, research post** и **API docs** для честного позиционирования M3 в README и demo copy. Подробный AI rubric в публично доступной submit-странице сейчас не раскрыт — там виден лишь заголовок и “Loading rubric...”, поэтому опирайтесь на 50/50 scoring, формулировки треков и артефакты репозитория.

²⁷

Итоговый вывод жёсткий, но честный: **если вы хотите именно максимизировать шанс победы, а не “показать амбицию”, то не стройте универсальную платформу ремонта всего на свете. Стройте узкий, безумно читаемый, доказуемый Lazarus MCP для Node/Python, сабмитьте его как The Resurrection, а MCP используйте как усиление, а не как центр тяжести.** Под правила арены это лучший trade-off между скоростью, новизной, judge-fit и голосуемостью. ³

¹ ⁸ ¹² ²¹ ²³ ²⁴ <https://arena.cysic.xyz/>

<https://arena.cysic.xyz/>

² ³ ⁵ ¹⁰ ¹¹ ¹³ ¹⁴ ¹⁶ ²² ²⁷ <https://arena.cysic.xyz/submit>

<https://arena.cysic.xyz/submit>

⁴ <https://arena.cysic.xyz/leaderboard>

<https://arena.cysic.xyz/leaderboard>

⁶ <https://docs.cysic.xyz/cysic-ai/cysic-automation/cyops/>

<https://docs.cysic.xyz/cysic-ai/cysic-automation/cyops/>

⁷ ²⁵ <https://docs.cysic.xyz/tutorial-docs/cyops-tutorial/how-to-use-cyops/>

<https://docs.cysic.xyz/tutorial-docs/cyops-tutorial/how-to-use-cyops/>

⁹ https://x.com/cysic_xyz/highlights

https://x.com/cysic_xyz/highlights

¹⁵ <https://www.minimax.io/models/text/m3>

<https://www.minimax.io/models/text/m3>

¹⁷ ¹⁸ <https://ts.sdk.modelcontextprotocol.io/>

<https://ts.sdk.modelcontextprotocol.io/>

¹⁹ https://ts.sdk.modelcontextprotocol.io/v2/documents/Documents.Server_Quickstart.html

https://ts.sdk.modelcontextprotocol.io/v2/documents/Documents.Server_Quickstart.html

²⁰ <https://ts.sdk.modelcontextprotocol.io/v2/>

<https://ts.sdk.modelcontextprotocol.io/v2/>

²⁶ https://x.com/cysic_xyz

https://x.com/cysic_xyz